

Chat IA para Sistema R

Especificación completa para construcción independiente por otro agente IA. Incluye contexto del dominio, esquema de datos, RPCs, herramientas, reglas de negocio, seguridad y plan de implementación.

IMPORTANTE: Este documento es una **especificación**. No modificar ni tocar el repositorio original de Sistema R. El Chat IA se construye en un repositorio separado.

1. Resumen del sistema

Sistema R es una plataforma web de gestión de actividades audiovisuales para **Rhino Audiovisuales**, una productora peruana que trabaja para el cliente **Aunor** bajo contrato anual. La plataforma gestiona el ciclo de vida completo de actividades de grabación, edición, creatividad y locución: planificación, asignación a operarios, ejecución, entrega de material, conversaciones internas, publicaciones para el cliente, confirmaciones y reemplazos.

Propiedad	Valor
Backend	Supabase (PostgreSQL 17 + Auth + Edge Functions)
Frontend	Next.js (App Router), React, TypeScript
Roles activos (2026)	admin, operario, aunor (Burson retirado)
Auth	Supabase Auth + JWT + sesiones propias (app_sessions) con expiración de 12h
Modo dual	demo (localStorage, local-only) / supabase (producción en Vercel)
Concurrencia	Versionado optimista (expected_version) en todas las mutaciones
Idempotencia	UUID key + SHA-256 hash en creación de actividades y operaciones Aunor
Preservación de datos	Triggers BEFORE DELETE bloquean borrado físico; solo erasure explícito con password puede eliminar
Auditoría	audit_events (actividades) + account_audit_events (cuentas) + record_history (triggers en todas las tablas)

2. Arquitectura del Chat IA

Principio: El Chat IA es un componente **independiente** con su propio repositorio. Se comunica con Sistema R exclusivamente a través de las RPCs y Edge Functions existentes, usando la identidad del usuario autenticado (JWT) para todas las operaciones.

2.1 Componentes

Capa	Responsabilidad	Tecnología
Widget de chat	Interfaz conversacional embebida en Sistema R (iframe o web component). Envía JWT del usuario + mensajes al Gateway. Renderiza respuestas streaming.	Web Component nativo (framework-agnostic) o React component. CSS personalizable vía variables.
Gateway	Valida JWT, extrae rol y user_id, construye system prompt dinámico, ejecuta tool calls contra Supabase RPCs, gestiona streaming y topes de gasto.	Edge Function de Supabase (Deno) o función serverless (Node/TypeScript). Preferir Deno por compatibilidad con el ecosistema Supabase.
Tool Layer	Declaración de herramientas estilo OpenAI function calling. Cada tool = 1 RPC autorizada o 1 vista consultada. El modelo selecciona qué tool llamar según la conversación.	JSON Schema para definición de tools. Formato compatible con OpenAI, Anthropic y OpenRouter.
Proveedor de modelo	OpenRouter con lista blanca de modelos económicos y potentes. Interfaz abstracta para cambiar de proveedor sin tocar el Gateway.	OpenRouter API. Interfaz propia ModelProvider con métodos chat(messages, tools).
Historial de chat	Persistencia de conversaciones por usuario. Particionado por proyecto+usuario. Retención configurable.	PostgreSQL (tabla propia en el proyecto Supabase de Sistema R) o Upstash Redis.

2.2 Flujo de llamada detallado

- Usuario autenticado** escribe un mensaje en el widget embebido.
- Widget** envía POST al Gateway: { messages: [...], jwt: "eyJ..." }. El JWT es el mismo que usa la sesión activa en Sistema R.
- Gateway** valida el JWT con Supabase Auth (supabaseAdmin.auth.getUser(jwt)). Extrae user_id, role, session_id.
- Gateway** verifica sesión activa llamando a private.has_active_app_session() vía RPC. Si no hay sesión válida, rechaza.
- Gateway** construye el **system prompt** según el rol del usuario. Para Admin: incluye todas las tools de escritura + lectura. Para Aunor: solo tools de consulta sobre vistas aunor_*.
- Gateway** envía al modelo: system prompt + historial de conversación + mensaje del usuario + declaración de tools disponibles.
- Modelo** decide: responde directamente (texto) o llama a una o más tools (paralelo o secuencial).
- Gateway** ejecuta cada tool llamando a la RPC correspondiente. Para consultas Aunor: usa supabaseClient con el JWT del usuario (RLS filtra). Para escrituras Admin: usa supabaseAdmin y pasa auth.uid() como actor para auditoría.
- Resultado** de cada tool se devuelve al modelo como tool_result.
- Modelo** formula respuesta final basada en los tool results + historial.
- Gateway** envía la respuesta al widget vía streaming, la guarda en el historial, y registra un evento de auditoría.

2.3 Contrato REST del Gateway

Endpoint	Método	Request Body	Response
----------	--------	--------------	----------

POST /v1/chat	POST	{ messages: ChatMessage[], jwt: string, stream?: boolean }	Streaming de chunks JSON: { type: "token" "tool_call" "tool_result" "done" "error", data: ... }
POST /v1/new-chat	POST	{ jwt: string }	{ chat_id: string } — inicia nueva conversación
GET /v1/chat/:id	GET	Header Authorization: Bearer JWT	{ messages: ChatMessage[] } — historial completo

ChatMessage({role: "user"|"assistant"|"system"|"tool", content: string, tool_calls?: ToolCall[], tool_call_id?: string, name?: string })

1 / 12

3. Base de datos — todas las tablas

La base de datos usa PostgreSQL 17 en Supabase. Schema `public` para tablas de negocio, schema `private` para tablas internas (sin acceso directo desde la app).

3.1 Enums

Enum	Valores	Uso
<code>public.app_role</code>	'operario', 'admin', 'burson', 'aunor'	Rol de cada perfil de usuario
<code>public.activity_status</code>	'Programada', 'En proceso', 'Entregada'	Estado del ciclo de vida de una actividad
<code>public.activity_type</code>	'Grabación', 'Edición', 'Creatividad', 'Locución'	Tipo de actividad audiovisual
<code>public.activity_origin</code>	'operario', 'burson'	Origen de la actividad (Burson retirado)

3.2 public.profiles

RLS: `SELECT policy — has_active_app_session() AND (id = auth.uid() OR current_app_role() = 'admin')`

Columna	Tipo	Default	Restricciones
<code>id</code>	<code>uuid</code>		PK → <code>auth.users(id)</code> ON DELETE RESTRICT
<code>username</code>	<code>citext</code>		UNIQUE. Regex: <code>^[a-z0-9](?:[a-z0-9._-]{0,62}[a-z0-9])?\$</code>
<code>display_name</code>	<code>text</code>		CHECK btrim between 2-120 chars
<code>role</code>	<code>app_role</code>		NOT NULL
<code>is_active</code>	<code>boolean</code>	<code>true</code>	NOT NULL
<code>is_burson_operator</code>	<code>boolean</code>	<code>false</code>	CHECK: solo true si <code>role=operario</code>
<code>can_create_own_activities</code>	<code>boolean</code>	<code>false</code>	CHECK: solo true si <code>role=operario AND is_active</code>
<code>must_change_password</code>	<code>boolean</code>	<code>false</code>	NOT NULL
<code>created_at</code>	<code>timestampz</code>	<code>now()</code>	
<code>updated_at</code>	<code>timestampz</code>	<code>now()</code>	

Índices únicos parciales: `profiles_one_active_burson_operator` (máx 1 operario burson activo), `profiles_one_active_burson` (máx 1 burson activo, retirado), `profiles_one_active_aunor` (máx 1 cuenta Aunor activa).

3.3 public.app_sessions

RLS: Sin políticas — default DENY. Solo se inserta/consulta desde RPCs SECURITY DEFINER.

Columna	Tipo	Default	Restricciones
<code>session_id</code>	<code>uuid</code>		PK
<code>user_id</code>	<code>uuid</code>		FK → <code>profiles(id)</code> ON DELETE CASCADE
<code>started_at</code>	<code>timestampz</code>	<code>now()</code>	NOT NULL
<code>expires_at</code>	<code>timestampz</code>	<code>now()+12h</code>	CHECK > <code>started_at</code> AND ≤ <code>started_at+12h</code>
<code>revoked_at</code>	<code>timestampz</code>		nullable

3.4 public.activities

RLS: `SELECT policy activities_select_authorized — can_view_activity(origin, created_by, responsible_id, deleted_at)`, que implementa: `admin=true`, `operario=own+not deleted`, `burson=own burson-origin (retirado)`, `else=false`.

Columna	Tipo	Default	Restricciones
<code>id</code>	<code>uuid</code>	<code>gen_random_uuid()</code>	PK
<code>origin</code>	<code>activity_origin</code>	'operario'	NOT NULL
<code>created_by</code>	<code>uuid</code>		FK → <code>profiles(id)</code> ON DELETE RESTRICT
<code>created_by_role</code>	<code>app_role</code>		NOT NULL
<code>responsible_id</code>	<code>uuid</code>		FK → <code>profiles(id)</code> ON DELETE RESTRICT
<code>responsible_name</code>	<code>text</code>		CHECK btrim 2-120
<code>type</code>	<code>activity_type</code>		NOT NULL
<code>title</code>	<code>text</code>		CHECK btrim 2-180
<code>description</code>	<code>text</code>		CHECK btrim 2-5000
<code>place</code>	<code>text</code>	''	CHECK ≤ 300
<code>status</code>	<code>activity_status</code>	'Programada'	NOT NULL
<code>material_link</code>	<code>text</code>	''	CHECK safe HTTPS URL o vacío
<code>operator_opinion</code>	<code>text</code>	''	CHECK ≤ 5000
<code>reference_link</code>	<code>text</code>	''	CHECK safe HTTPS URL o vacío

version	integer	1	CHECK > 0. Se incrementa en cada mutación.
idempotency_key	uuid		nullable, UNIQUE con created_by
idempotency_hash	text		nullable, CHECK sha256 hex
thread_opened_at	timestamptz		nullable. Se setea al primer mensaje de conversación
delivered_at	timestamptz		nullable. Se setea al llegar a Entrega
deleted_at	timestamptz		nullable (soft delete)
deleted_by	uuid		FK → profiles(id), nullable
deletion_reason	text		nullable, CHECK 2-1000
created_at	timestamptz	now()	
updated_at	timestamptz	now()	

3.5 public.activity_date_spans

Columna	Tipo	Default	Restricciones
id	bigint	generated always as identity	PK
activity_id	uuid		FK → activities(id) ON DELETE CASCADE
position	smallint		CHECK 1-100, UNIQUE por activity_id
start_date	date		NOT NULL, ≥ 2026-01-01
end_date	date		NOT NULL, ≥ start_date, ≤ start_date+3660 días
place	text	' '	CHECK ≤ 300 (añadido sep 2026)

3.6 public.audit_events

Col.	Tipo	Descripción
id	bigint GENERATED ALWAYS AS IDENTITY	PK
activity_id	uuid	FK → activities(id) ON DELETE RESTRICT
actor_id	uuid	FK → profiles(id)
actor_name	text	Nombre del actor
actor_role	app_role	Rol del actor
action	text	CHECK 2-180. Ej: 'Actividad creada', 'Estado avanzado a Entregada'
detail	jsonb	Datos adicionales. Default '{}'
created_at	timestamptz	Default now()

3.7 public.account_audit_events

Campos: id (bigint PK), target_profile_id (FK profiles), actor_id, actor_name, action, detail (jsonb), created_at. Como audit_events pero para acciones sobre cuentas.

3.8 public.activity_messages (conversación interna Admin–Operario)

Columna	Tipo	Restricciones
id	uuid PK	Default gen_random_uuid()
activity_id	uuid	FK → activities(id) ON DELETE RESTRICT
author_id	uuid	FK → profiles(id)
author_name	text	CHECK 2-120
author_role	app_role	CHECK IN ('admin','operario')
body	text	CHECK btrim 1-5000
opens_thread	boolean	CHECK solo admin puede abrir. Único por actividad.
version	integer	CHECK > 0, default 1
created_at	timestamptz	Default now()
edited_at	timestamptz	nullable, CHECK ≥ created_at
deleted_at	timestamptz	nullable (soft delete)
deleted_by	uuid	CHECK = author_id (solo autor puede borrar)

RLS: activity_messages_select_participants — deleted_at IS NULL AND can_access_activity_conversation(activity_id)

Reglas de negocio de la conversación:

- Solo actividades en estado Entregada pueden tener conversación.
- Admin debe iniciar el hilo (primer mensaje con opens_thread=true).
- Al abrir el hilo, material_link y operator_opinion se congelan (no se pueden modificar).
- Participantes: Admin (todos los mensajes) + Operario responsable (solo sus mensajes).
- Edición y borrado: solo del propio autor. El borrado es soft-delete (visible para Admin como evento pero cuerpo oculto).

3.9 Vistas públicas para Aunor (security_barrier, security_invoker=false)

Vista	Qué contiene	Filtro RLS
public.aunor_activities	Activities + aunor_publications. Actividades publicadas para Aunor.	can_access_aunor_activity()
public.aunor_journeys	activity_date_spans de actividades Aunor.	can_access_aunor_activity()
public.aunor_services	private.aunor_services (12 servicios precargados).	Solo admin/aunor
public.aunor_deliveries	Deliveries + confirmations. Incluye is_current computado.	can_access_aunor_activity()
public.aunor_agreements	Acuerdos registrados. recorded_by mostrado como 'Admin · Rhino'.	can_access_aunor_activity()
public.aunor_replacements	Reemplazos + confirmations. Incluye is_current.	can_access_aunor_activity()

public.aunor_messages	Mensajes externos Aunor–Admin. author mapeado a display label.	can_access_aunor_activity()
-----------------------	--	-----------------------------

3.10 Tablas en schema private (sin acceso directo desde la app)

Tabla	Propósito	Campos clave
private.aunor_services	Catálogo de servicios Aunor (12 filas precargadas)	id (text PK), position, label, reference ('Cláusula 2.2')
private.aunor_publications	Publicaciones de actividades para Aunor	activity_id, version, summary, service_id, superseded_at
private.aunor_material_revisions	Contador de revisiones de material_link	activity_id PK, revision integer default 0
private.aunor_deliveries	Entregas de material a Aunor	activity_id, version, material_revision, material_link, label, published_by
private.aunor_agreements	Acuerdos registrados (llamada/reunión/verbal)	activity_id, channel, contacted_at, requester_declared, body, evidence_link, recorded_by
private.aunor_replacements	Reemplazos de actividad (original ↔ sustituto)	original_activity_id, substitute_activity_id, agreement_id, reason, evidence
private.aunor_confirmations	Confirmaciones de entregas o reemplazos	delivery_id (XOR), replacement_id (XOR), confirmed_by
private.aunor_messages	Mensajes externos (solo admin/aunor)	activity_id, author_role, body, sequence (identity)
private.aunor_read_markers	Marcadores de lectura por usuario	activity_id + profile_id (PK), last_sequence
private.aunor_requests	Idempotencia de mutaciones Aunor	actor_id + request_id (PK), fingerprint, result
private.credential_operations	Mutex para operaciones de credenciales	profile_id PK, operation_id, actor_id
private.account_session_boundaries	Límite de sesión por cambio de perfil	profile_id PK, valid_after timestamptz
private.record_history	Preservación histórica de cambios (trigger)	table_name, operation, old_record jsonb, new_record jsonb
private.erasure_context	Contexto de borrado físico (transacción activa)	transaction_id PK, manifest name
private.erasure_account_requests	Solicitudes de borrado de cuenta	target_id PK, session_id, fingerprint, expires_at
private.erasure_password_attempts	Rate limiting para borrado (5 intentos/15min)	actor_id PK, window_start, attempts

4. RPCs del sistema (PostgreSQL via Supabase)

Todas las RPCs están definidas en migraciones SQL en `supabase/migrations/`. Se acceden desde el Gateway del Chat IA mediante `supabase.rpc(nombre, params)`.

4.1 RPCs de sesión y autenticación

RPC	Params	Return	Qué hace	RoI
register_app_session()	ninguno	void	Registra la sesión de Supabase Auth en app_sessions con user_id y session_id del JWT. Crea o ignora si ya existe (ON CONFLICT DO NOTHING).	cualquier authenticated
revoke_current_app_session()	ninguno	void	Marca revoked_at en la sesión actual.	cualquier authenticated
access_directory_v1()	ninguno	TABLE(username, display_name, role)	Lista de perfiles activos para el login picker. Granted a anon + authenticated.	público

4.2 RPCs de actividades — Admin planifica

RPC	Params	Return	Reglas
plan_activity_v2(p_idempotency_key, p_responsible_id, p_type, p_title, p_description, p_place, p_spans)	7 params	(activity_id uuid, activity_version int, replayed boolean)	Crea actividad con origen operario, asignada a un operario. Valida payload con assert_activity_payload. Idempotente por key+hash. Llama replace_activity_spans para insertar las jornadas. Solo Admin.
replan_activity_v2(p_activity_id, p_expected_version, p_responsible_id, p_type, p_title, p_description, p_place, p_spans)	8 params	(activity_id uuid, activity_version int)	Actualiza planificación de actividad existente. Verifica expected_version = current version. Reemplaza spans atómicamente. Solo Admin.

4.3 RPCs de actividades — Operario ejecuta

RPC	Params	Return	Reglas
create_own_activity_v2(p_idempotency_key, p_type, p_title, p_description, p_place, p_spans)	6 params	(activity_id uuid, activity_version int, replayed boolean)	Operario crea su propia actividad. Requiere can_create_own_activities=true.
update_execution_v1(p_activity_id, p_expected_version, p_material_link, p_operator_opinion)	4 params	(activity_id uuid, activity_version int)	Operario actualiza material_link y opinión. Solo si el hilo de conversación NO está abierto (thread_opened_at IS NULL).
advance_activity_v1(p_activity_id, p_expected_version)	2 params	(activity_id, activity_version, activity_status)	Programada → En proceso → Entregada. Para llegar a Entregada, material_link debe ser válido. El operario responsable ejecuta.

4.4 RPCs de administración de actividades

RPC	Params	Return	Reglas
soft_delete_activity_v1(id, expected_version, reason)	3 params	(activity_id, version, deleted_at)	Baja lógica. Setea deleted_at, deleted_by, deletion_reason. Solo Admin.
restore_activity_v1(id, expected_version, p_responsible_id=null)	3 params	(activity_id, version, responsible_id)	Restaura desde papelera. Si la actividad estaba abierta (sin entregar), exige responsible_id activo. Solo Admin.
reset_activity_v1(id, expected_version, reason)	3 params	(activity_id, version)	Vuelve a Programada. Conserva material, opinión, conversación y evidencia. Solo Admin.
restart_activity_v2(id, expected_version, reason)	3 params	(activity_id, version)	Crea copia fresca de la actividad y archiva la original en papelera. Solo Admin, solo actividades de origen operario.
set_operator_creation_permission_v1(operator_id, enabled)	2 params	(profile_id, can_create_own_activities)	Concede o revoca permiso de creación propia a un Operario. Solo Admin.

4.5 RPCs de conversación interna

RPC	Params	Return	Reglas
post_activity_message_v1(activity_id, expected_version, body)	3 params	(activity_id, activity_version, message_id, message_version, opened_at)	Admin abre el hilo (si no estaba abierto). Operario responde (si ya está abierto). expected_version puede ser null para ignorar verificación de concurrencia.
edit_activity_message_v1(message_id, expected_version, body)	3 params	(activity_id, version, message_id, message_version)	Edita mensaje propio. Verifica expected_version.
delete_activity_message_v1(message_id, expected_version)	2 params	(activity_id, version, message_id, message_version)	Soft-delete de mensaje propio.

4.6 RPC universal de Aunor

RPC	Params	Return	Comandos
aunor_mutate_v1(p_command, p_activity_id, p_request_id, p_payload)	4 params	jsonb	Comandos: publish (Admin), delivery (Admin), agreement (Admin), replacement (Admin), confirm-delivery (Aunor), confirm-replacement (Aunor), read (retirado), message (retirado). Idempotente vía aunor_requests.

Estructura de p_payload por comando:

- publish: { service_id, summary, not_performed_reason? }
- delivery: { material_link, label } (material_link debe ser HTTPS seguro)
- agreement: { channel, contacted_at, requester_declared, body, evidence_link? }
- replacement: { original_activity_id, substitute_activity_id, reason, evidence_note, evidence_link? }
- confirm-delivery: { delivery_id } — solo Aunor
- confirm-replacement: { replacement_id } — solo Aunor

p_request_id debe ser un UUID único por operación para idempotencia.

4.7 RPCs de cuentas (llamadas desde Edge Functions, no directamente)

RPC	Params	Return	Quién la llama
create_account_profile_v1(profile_id, username, display_name, role, is_burson_operator, can_create_own_activities, actor_id)	7 params	(profile_id, profile_updated_at)	Edge Function admin-accounts (service_role)
update_account_v1(profile_id, expected_updated_at, display_name, role, is_active, is_burson_operator, can_create_own_activities)	7 params	(profile_id, profile_updated_at)	Authenticated (Admin). Se ejecuta con JWT del usuario.
prepare_temporary_password_reset_v1(profile_id, actor_id)	2 params	(profile_id)	Edge Function admin-accounts (service_role)
confirm_temporary_password_reset_v1(profile_id, actor_id)	2 params	(profile_id)	Edge Function admin-accounts (service_role)
complete_temporary_password_change_v1(profile_id)	1 param	(profile_id)	Edge Function change-temporary-password (service_role)

4.8 RPCs de borrado físico (erasure)

RPC	Params	Return
erasure_identity_v1()	ninguno	jsonb: {actorId, sessionId}
preview_erasure_v1(kind, target=null)	2 params	jsonb: {fingerprint, total, activities[], counts {table: n}}
begin_erasure_password_attempt_v1(actor, session)	2 params	boolean (rate-limited, 5/15min)
execute_erasure_v1(actor, session, kind, target, fingerprint)	5 params	jsonb
cancel_erasure_v1(actor, target)	2 params	void

Reglas de negocio del erasure:

- Requiere Admin autenticado + re-verificación de password (5 intentos máx en ventana de 15 min).
- kind = 'trash' (vacía papelera) o 'account' (elimina cuenta inactiva + datos).
- Confirmación textual requerida: "ELIMINAR DEFINITIVAMENTE".
- Builds un manifest con todas las filas FK-descendientes antes de borrar. Si el manifest cambia entre preview y execute, la operación falla.
- Los triggers BEFORE DELETE de todas las tablas permiten el borrado solo cuando hay un erasure_context activo.

5. Edge Functions

Tres funciones desplegadas en Supabase Edge Runtime (Deno). El Chat IA NO necesita reimplementarlas — las consume via fetch a `SUPABASE_URL/functions/v1/{name}`.

5.1 admin-accounts

Propiedad	Valor
URL	/functions/v1/admin-accounts
Método	POST
Auth	withSupabase({ auth: "user" }) — JWT de Supabase Auth + perfil Admin verificado

Acción: create

- Body: { action: "create", username, display_name, role, temporary_password, is_burson_operator?, can_create_own_activities? }
- Flujo: Valida username/rol → Crea Auth user con `supabaseAdmin.auth.createUser({email, password, email_confirm:true})` → Llama `create_account_profile_v1` RPC → Si falla el RPC, revierte el Auth user
- Error: Si el Auth user ya existe (idempotencia), lo recupera y continúa

Acción: reset-password

- Body: { action: "reset-password", profile_id, temporary_password }
- Flujo: Adquiere mutex `credential_operations` → `prepare_temporary_password_reset_v1` → Auth `updateUserById(password)` + `updateUserById(app_metadata)` con fingerprint PBKDF2 → `confirm_temporary_password_reset_v1`

5.2 change-temporary-password

Propiedad	Valor
URL	/functions/v1/change-temporary-password
Método	POST
Auth	withSupabase({ auth: "user" }) — JWT del usuario que debe cambiar clave

- Body: { password: "nuevaClave123!" }
- Flujo: Verifica `must_change_password=true` → Compara PBKDF2 fingerprint del JWT vs nueva password (rechaza si son iguales) → Auth `updateUserById(password)` → `complete_temporary_password_change_v1` → limpia fingerprint de `app_metadata`

5.3 admin-erasure

Propiedad	Valor
URL	/functions/v1/admin-erasure
Método	POST
Auth	withSupabase({ auth: "user" }) — JWT de Admin

- Body: { kind: "trash"|"account", target: "uuid"|null, password: "...", fingerprint: "hex...", confirmation: "ELIMINAR DEFINITIVAMENTE" }
- Flujo: `erasure_identity_v1()` → `begin_erasure_password_attempt_v1()` (rate limit) → `ephemeral signInWithPassword` para verificar password Admin → `execute_erasure_v1()` → si `kind=account`: `auth.admin.deleteUser(target)` → `cancel_erasure_v1()`

6. Funciones helper de seguridad (schema private)

Estas funciones **no se pueden llamar desde el Chat IA** directamente (revoke all). El Gateway debe llamar a las RPCs públicas que internamente invocan estas helpers.

Función	Propósito	Cómo se usa en el Chat IA
<code>private.has_active_app_session()</code>	Verifica que el JWT tiene una sesión <code>app_sessions</code> activa y no expirada.	Se llama automáticamente en cada RPC pública (está en las políticas RLS y en los guards de las RPCs).
<code>private.current_app_role()</code>	Devuelve el rol del perfil asociado al JWT.	Idem. Se usa en todas las políticas RLS.
<code>private.can_access_aunor_activity(uuid)</code>	Verifica que el usuario puede ver una actividad publicada para Aunor.	Se usa en las vistas <code>aunor_*</code> — se aplica automáticamente al consultar.
<code>private.can_access_activity_conversation(uuid)</code>	Verifica que el usuario puede ver la conversación interna de una actividad.	Se usa en RLS de <code>activity_messages</code> .
<code>private.is_safe_https_url(text)</code>	Valida que una URL sea HTTPS sin credenciales embebidas.	Se usa automáticamente en los CHECK constraints de <code>activities</code> y <code>aunor_deliveries</code> .
<code>private.require_operator_profile()</code>	Verifica que el JWT corresponde a un operario activo.	Se usa en RPCs de operario. Si el Chat IA llama como Admin a una RPC de operario, fallará.
<code>private.require_admin_profile()</code>	Verifica que el JWT corresponde a un Admin activo.	Se usa en RPCs de Admin. El Chat IA Admin debe llamar a estas RPCs con el JWT del Admin.

 **Regla de seguridad crítica:** El Gateway del Chat IA nunca debe usar `service_role` para leer datos. Para consultas, usar `supabaseClient` con el JWT del usuario (las RLS filtran automáticamente). Para escrituras Admin, usar `supabaseAdmin` solo porque las RPCs llaman a `auth.uid()` internamente — el

7. Tool Layer — Catálogo completo

Cada tool se define como JSON Schema para OpenAI/Anthropic function calling. El Gateway las declara en el cuerpo del request al modelo según el rol del usuario.

7.1 Tools para Admin — Lectura (14 tools)

#	Nombre de la tool	Qué hace	RPC / Query Supabase	Inputs
1	list_activities	Lista actividades según filtros	supabase.from('activities').select().eq('deleted_at',null).match(filters)	status?, type?, responsible_id?, date_from?, date_to?
2	get_activity_detail	Ficha completa de una actividad + spans + auditoría	Activities by id + activity_date_spans + audit_events	activity_id: uuid
3	get_activity_messages	Conversación interna de una actividad entregada	supabase.from('activity_messages').select().eq('activity_id',id).is('deleted_at',null)	activity_id: uuid
4	list_operators	Lista operarios activos disponibles	supabase.from('profiles').select().eq('role','operario').eq('is_active',true)	(ninguno)
5	list_accounts	Lista todas las cuentas (todos los roles)	supabase.from('profiles').select('*') (RLS permite solo a Admin ver todas)	role?, is_active?
6	get_account_detail	Detalle de cuenta + auditoría de cambios	Profile by id + account_audit_events	profile_id: uuid
7	get_historical_year	Actividades de un año histórico	supabase.rpc('list_supabase_historical_activities', {year}) — lógica de spans overlapping	year: integer (≥2026)
8	get_trash	Lista actividades en papelera	supabase.from('activities').select().is('deleted_at','not.null')	(ninguno)
9	get_audit_trail	Auditoría de una actividad	supabase.from('audit_events').select().eq('activity_id',id)	activity_id: uuid
10	search_activities	Búsqueda textual en título y descripción	supabase.from('activities').select().or('title.ilike.%query%,description.ilike.%query%')	query: string (mín 3 chars)
11	list_aunor_publications	Actividades publicadas para Aunor (visión Admin)	supabase.from('aunor_activities').select('*') (vista security_barrier)	filtros opcionales
12	get_aunor_deliveries	Entregas Aunor de una actividad	supabase.from('aunor_deliveries').select().eq('activity_id',id)	activity_id: uuid
13	get_aunor_agreements	Acuerdos registrados para una actividad	supabase.from('aunor_agreements').select().eq('activity_id',id)	activity_id: uuid
14	get_aunor_replacements	Reemplazos registrados para una actividad	supabase.from('aunor_replacements').select().eq('activity_id',id)	activity_id: uuid

7.2 Tools para Admin — Escritura (14 tools)

Todas requieren confirmación explícita de 2 pasos.

#	Nombre	RPC	Inputs	Notas
15	create_activity	plan_activity_v2	type, title, description, place?, responsible_id, spans[] (array de {start, end, place?})	Genera idempotency_key y hash automáticamente. type debe ser uno de: Grabación, Edición, Creatividad, Locución.
16	replan_activity	replan_activity_v2	activity_id, expected_version, type, title, description, place?, responsible_id,	Requiere leer version actual antes de llamar.

			spans[]	
17	advance_activity	advance_activity_v1	activity_id, expected_version	Avanza un estado: Programada→En proceso→Entregada. Solo si el operario ya cargó material_link.
18	update_activity_execution	update_execution_v1	activity_id, expected_version, material_link?, operator_opinion?	Solo si hilo de conversación no está abierto. Alternativamente, Admin delega al operario.
19	soft_delete_activity	soft_delete_activity_v1	activity_id, expected_version, reason (2-1000 chars)	Baja a papeleria. Siempre preguntar motivo.
20	restore_activity	restore_activity_v1	activity_id, expected_version, new_responsible_id?	Restaura desde papeleria. Si actividad estaba abierta, pedir responsable_id.
21	reset_activity	reset_activity_v1	activity_id, expected_version, reason	Vuelve a Programada. Conserva material, conversación y auditoría.
22	restart_activity	restart_activity_v2	activity_id, expected_version, reason	Copia fresca + archiva original. Solo operario-origin.
23	post_internal_message	post_activity_message_v1	activity_id, expected_version?, body	Envía mensaje en conversación interna. Si es el primero, abre el hilo.
24	publish_aunor_activity	aunor_mutate_v1 command='publish'	activity_id, request_id, payload: {service_id, summary}	Publica actividad para visibilidad Aunor.
25	create_aunor_delivery	aunor_mutate_v1 command='delivery'	activity_id, request_id, payload: {material_link, label}	Crea entrega de material para Aunor.
26	register_aunor_agreement	aunor_mutate_v1 command='agreement'	activity_id, request_id, payload: {channel, contacted_at, requester_declared, body}	Registra acuerdo por llamada/reunión/verbal.
27	create_aunor_replacement	aunor_mutate_v1 command='replacement'	payload: {original_activity_id, substitute_activity_id, reason, evidence_note}	Registra reemplazo de actividad.
28	set_creation_permission	set_operator_creation_permission_v1	operator_id, enabled (boolean)	Concede/revoca permiso de autocreación a operario.

7.3 Tools para Aunor — Solo lectura (8 tools)

#	Nombre	Vista Supabase	Inputs	Qué devuelve
A1	my_activities	aunor_activities	status_filter?, type_filter?	Lista de actividades publicadas para Aunor, con estado, fechas, código (GR-021, ED-009, etc.)
A2	my_activity_detail	aunor_activities + aunor_journeys + aunor_deliveries	activity_id: uuid	Ficha completa: datos base, jornadas, entregas, confirmaciones
A3	my_deliveries	aunor_deliveries	activity_id?	Entregas de material con estado de confirmación
A4	my_agreements	aunor_agreements	activity_id?	Acuerdos registrados (llamadas, reuniones)
A5	my_replacements	aunor_replacements	activity_id?	Reemplazos de actividad
A6	my_messages	aunor_messages	activity_id: uuid	Mensajes del hilo externo Aunor-Admin
A7	search_media	aunor_deliveries (búsqueda en label + material_link)	query: string, date_from?, date_to?	Entregas cuyo label o link contienen el texto buscado
A8	calendar_journeys	aunor_journeys	year: integer, month?: integer	Jornadas agrupadas por actividad para visualización calendario

CRÍTICO para Aunor: Las consultas Aunor deben usar `supabaseClient` con el **JWT del usuario Aunor**, NO `supabaseAdmin`. Las vistas `aunor_*` tienen `security_barrier` y filtran por `can_access_aunor_activity()`. Si se usa `service_role`, se rompe el aislamiento y la IA podría ver datos no autorizados.

8. Implementación del Gateway

8.1 Gestión de la sesión del Chat IA

El Gateway debe validar que el JWT del usuario corresponde a una sesión activa en Sistema R antes de procesar cualquier mensaje:

1. Recibir `jwt` del widget.
2. Llamar a Supabase Auth: `supabaseAdmin.auth.getUser(jwt)` para validar el token y obtener `user_id`.
3. Llamar a `supabase.rpc('register_app_session')` — si falla con `SR002`, la sesión no existe o expiró. Rechazar el mensaje.
4. Extraer `role` del perfil (query profiles por `user_id`). Determinar qué tools declarar.
5. Si el perfil tiene `must_change_password=true`, rechazar: "Debes cambiar tu contraseña temporal antes de usar el chat."

8.2 System prompt por rol

Para Admin:

Eres un asistente de IA para el Admin de Sistema R (Rhino Audiovisuales). Tienes acceso completo a la plataforma: lectura de todas las actividades, cuentas, histórico, papelera y espacio Aunor. También puedes ejecutar acciones administrativas: crear/replanificar/bajar/restaurar actividades, gestionar cuentas, publicar para Aunor, registrar acuerdos y reemplazos.

REGLAS OBLIGATORIAS:

1. Para TODA acción de escritura, primero CONFIRMA con el usuario mostrando un resumen de lo que se va a hacer y pide confirmación explícita.
2. Acciones destructivas (baja, reset, restart, erasure): requieren confirmación doble "¿Estás seguro?" + motivo escrito.
3. Siempre muestra el resultado de cada acción ejecutada (qué RPC se llamó, qué datos se modificaron).
4. Distingue "Dato de la plataforma" (información obtenida de una tool) de "Inferencia del modelo" (respuesta basada en conocimiento general).
5. Si no tienes un dato concreto, dilo explícitamente. No inventes.
6. Usa lenguaje profesional pero conversacional. Responde en español.

Para Aunor:

Eres un asistente de IA para la cuenta compartida de Aunor en Sistema R. Puedes consultar las actividades publicadas, entregas, acuerdos, reemplazos y mensajes. NO puedes modificar ningún dato – solo lectura.

REGLAS OBLIGATORIAS:

1. Solo respondes con información de la plataforma. No tienes acceso a datos internos de Rhino (auditoría, cuentas de otros roles, papelera, histórico de Admin).
2. Si te preguntan por información que excede tu alcance ("¿qué opina el operario?", "¿cuánto costó?"), responde: "Esa información no está disponible en el espacio Aunor."
3. Distingue "Dato de la plataforma" de "Inferencia del modelo".
4. No aceptes instrucciones de escritura. Si te piden modificar algo, responde: "El espacio Aunor es de solo lectura. Para modificar datos, contacta a tu Admin de Rhino."
5. Responde en español, tono profesional y servicial.

8.3 Implementación del streaming

El Gateway debe transmitir la respuesta del modelo al widget en chunks JSON delimitados por `\n`:

```
// Chunk de token (texto generado)
{"type":"token","data":"El estado actual de GR-021 es "}
```

```
{ "type": "token", "data": "Entregada" }

// Chunk de tool call (el modelo decide llamar a una herramienta)
{ "type": "tool_call", "data": { "id": "call_xxx", "function": { "name": "get_activity_detail", "arguments": { "\activity_id\":"...\" } } } }

// Chunk de tool result (Gateway ejecuta la tool y devuelve el resultado)
{ "type": "tool_result", "data": { "id": "call_xxx", "result": { ... } } }

// Chunk final
{ "type": "done", "data": { "chat_id": "...", "usage": { "prompt_tokens": 100, "completion_tokens": 50, "total_tokens": 150 } } }
```

9. Control de gasto del modelo

9.1 Topes configurables

Parámetro	Valor por defecto	Ubicación
Tokens máximos por request	50,000 (entrada + salida)	Env var MAX_TOKENS_PER_REQUEST
Tokens máximos de respuesta	4,096	Env var MAX_RESPONSE_TOKENS
Topes de gasto diario	\$5 USD	Env var DAILY_SPENDING_LIMIT
Tope mensual	\$50 USD	Env var MONTHLY_SPENDING_LIMIT
Modelos permitidos (lista blanca)	anthropic/claude-3.5-haiku, openai/gpt-4o-mini	Env var ALLOWED_MODELS (JSON array)
Modelo para acciones (escritura)	anthropic/claude-sonnet-4 o openai/gpt-4o	Env var ACTION_MODEL

9.2 Implementación de topes

- Tokens/request:** Contar tokens del prompt + historial antes de enviar al modelo. Rechazar si excede.
- Gasto diario:** Almacenar en una tabla chat_usage: {date, user_id, tokens_in, tokens_out, cost, model}. Sumar coste del día antes de cada request.
- Lista blanca:** Validar que el modelo seleccionado esté en ALLOWED_MODELS. Si no, usar el modelo por defecto.
- Modelo económico vs potente:** Para consultas simples (lectura), usar modelo económico (Haiku, GPT-4o-mini). Para acciones de escritura, usar modelo potente (Sonnet, GPT-4o). El Gateway decide según si la última tool llamada fue de escritura.

10. Modelo de seguridad detallado

10.1 Principios

- La IA nunca tiene más permisos que el usuario cuyo JWT usa.** No hay "cuenta de IA" con superpoderes.
- El Gateway nunca usa service_role para consultar datos del producto.** Service_role solo para operaciones Auth (crear usuario, actualizar password) y para RPCs que internamente verifican el actor.
- Las herramientas de escritura siempre requieren confirmación explícita.** La IA debe mostrar un resumen de la acción y esperar "Sí" o "Confirmo" antes de ejecutar.
- Las herramientas destructivas requieren doble confirmación + motivo.** El sistema debe registrar el motivo en el campo correspondiente.
- Toda acción de escritura debe registrarse en audit_events.** Si la RPC no lo hace automáticamente (como las RPCs de actividades), el Gateway debe insertar un evento de auditoría.
- El historial de chat se particiona por usuario.** Un Admin no puede ver conversaciones de otro Admin.

10.2 Matriz de riesgos y mitigaciones

Riesgo	Prob.	Impacto	Mitigación
IA ejecuta acción no deseada	Media	Alto	Confirmación de 2 pasos feature flag para desactivar tools de escritura (CHAT_IA_WRITE_ENABLED=false). Auditoría obligatoria post-ejecución.
Fuga de datos entre roles (Admin ve datos Aunor o viceversa)	Baja	Crítico	RLS en todas las consultas (JWT del usuario). Las tools Aunor usan vistas security_barrier. Tests de integración de aislamiento.
Prompt injection — usuario engaña al modelo para ejecutar acciones	Alta	Medio	Las tools devuelven datos estructurados (JSON), no texto libre. System prompt instruye ignorar instrucciones de escritura en el mensaje del usuario. Tools de escritura siempre requieren confirmación.
Gasto excesivo en API	Media	Medio	Topes día/mes, tokens/request, modelos baratos por defecto. Alertas configurables.
Alucinaciones (modelo inventa datos)	Alta	Medio	Cada respuesta distingue "Dato de la plataforma" vs "Inferencia". Las afirmaciones sin tool result se etiquetan como inferencia.
Clave API expuesta	Baja	Alto	OPENROUTER_API_KEY solo en el Gateway (variable de entorno). Nunca en código, logs, ni respuesta al widget.
Denegación de servicio (abuso del chat)	Media	Medio	Rate limiting por usuario: N requests/minuto. Topes de gasto compartidos. Rechazar si el usuario no tiene sesión activa.

10.3 Esquema de la tabla de uso

Tabla recomendada: private.chat_usage (en el mismo proyecto Supabase de Sistema R, schema private):

Columna	Tipo	Restricciones
id	bigint GENERATED ALWAYS AS IDENTITY	PK
user_id	uuid	FK → profiles(id)
chat_id	uuid	FK → chat_sessions(id)
model	text	Ej: "claude-3.5-haiku"
tokens_in	integer	
tokens_out	integer	
cost_usd	numeric(10,6)	
tool_calls	integer	Default 0
created_at	timestampz	Default now()

10.4 Esquema de tabla de sesiones de chat

Tabla: private.chat_sessions:

Columna	Tipo	Restricciones
id	uuid	PK, default gen_random_uuid()
user_id	uuid	FK → profiles(id)
title	text	Default '' — resumen automático
tool_calls_count	integer	Default 0
created_at	timestampz	Default now()
updated_at	timestampz	Default now()

Tabla: private.chat_messages:

Columna	Tipo	Restricciones
id	bigint GENERATED ALWAYS AS IDENTITY	PK
chat_id	uuid	FK → chat_sessions(id) ON DELETE CASCADE
role	text	'user', 'assistant', 'tool'
content	text	NOT NULL
tool_calls	jsonb	nullable
tool_call_id	text	nullable
name	text	nullable (tool name)
created_at	timestampz	Default now()

11. Plan de implementación por fases

Fase 0 — Contrato y evaluación

Duración estimada: 1 semana

Sin conexión a Sistema R.

- Crear repositorio `sistema-r-chat-ia` (vacío, sin código de Sistema R).
- Definir 20-30 preguntas reales por rol con respuestas esperadas (golden set).
- Ejemplo Admin: "Crea una actividad de Grabación para Ana Torres, título 'Cobertura Norte', del 5 al 7 de octubre."
- Ejemplo Aunor: "¿Qué actividades tengo publicadas en junio? Muéstrame las entregas."
- Configurar cuenta OpenRouter con los USD 10 disponibles.
- Evaluar Claude Haiku 3.5 y GPT-4o mini contra el golden set. Medir precisión, latencia, coste.
- Documentar resultado de evaluación y modelo(s) seleccionados.

Fase 1 — Gateway + Tool Layer MVP

Duración estimada: 2-3 semanas

Resultado: Chat IA funcional para Admin y Aunor con 5 tools de lectura cada uno.

1. **Gateway:** Edge Function en Supabase (Deno) o serverless Node. Endpoint `POST /v1/chat` con validación JWT, rol extraction, streaming.
2. **System prompts** según rol (copiar los del capítulo 8).
3. **Tools Admin lectura (5):** `list_activities`, `get_activity_detail`, `list_operators`, `search_activities`, `get_historical_year`.
4. **Tools Aunor (5):** `my_activities`, `my_activity_detail`, `my_deliveries`, `search_media`, `calendar_journeys`.
5. **Widget:** Web Component o React component que envía JWT + mensajes y renderiza streaming. Integrarlo en Sistema R con feature flag `CHAT_IA_ENABLED`.
6. **Control de gasto:** Implementar topes tokens/request, USD/día, USD/més. Tabla `chat_usage`.
7. **Integración en Sistema R:** Botón flotante "Chat IA" visible solo para Admin y Aunor (según `currentRole()`). Abre panel lateral con el widget.

Fase 2 — Tools de escritura Admin

Duración estimada: 2-3 semanas

Resultado: Admin puede ejecutar acciones administrativas por chat.

1. **Tools:** `create_activity`, `replan_activity`, `advance_activity`, `soft_delete_activity`, `restore_activity`, `reset_activity`, `post_internal_message`.
2. **Confirmación de 2 pasos:** La IA siempre muestra resumen y pide confirmación antes de ejecutar.
3. **Lectura de versión:** Antes de mutar, la tool debe leer el `expected_version` actual de la actividad.
4. **Auditoría:** Cada tool de escritura registra en `audit_events` la acción realizada por el Chat IA. Incluir en `detail`: `{"source": "chat-ia", "tool": "create_activity", ...}`.
5. **Idempotencia:** `create_activity` genera automáticamente `idempotency_key` (UUID) y `idempotency_hash` (SHA-256 del payload).

Fase 3 — Tools Aunor completas + acciones Aunor

Duración estimada: 1-2 semanas

1. **Tools Aunor restantes:** `my_agreements`, `my_replacements`, `my_messages`.
2. **Confirmación Aunor:** Aunor puede confirmar entregas y reemplazos a través del chat (única acción de escritura permitida). La tool `confirm_delivery` llama a `aunor_mutate_v1` `command='confirm-delivery'`.
3. **Verificación de RLS:** Test de integración que verifica que un usuario Aunor NO puede ver datos internos ni ejecutar tools de escritura. Usar 2 JWT diferentes (admin + aunor) contra el mismo Gateway.

Fase 4 — Confianza, UX y producción

Duración estimada: 2 semanas

1. **Marcado de fuentes:** Cada respuesta muestra qué datos vienen de la plataforma (icono 📄) vs inferencia del modelo (💡).
2. **Transparencia de tools:** El widget muestra qué tool se llamó y qué datos se consultaron (inspector expandible).
3. **Persistencia de conversación:** Las tablas `chat_sessions` y `chat_messages` implementadas. El usuario puede retomar conversaciones anteriores.
4. **Rate limiting:** Máximo 30 requests/minuto por usuario. Implementar con tabla `chat_rate_limits` o middleware.
5. **Feature flags:** `CHAT_IA_ENABLED` (booleano), `CHAT_IA_WRITE_ENABLED` (booleano, seguridad adicional).
6. **Pruebas:** Tests unitarios del Gateway, tests de integración con RPCs reales, tests de aislamiento de roles, tests de límites de gasto.
7. **Evaluación continua:** Ejecutar golden set semanalmente, reportar precisión.

Fase 5 — Reutilización (futuro)

Duración estimada: 2 semanas (cuando se requiera)

1. SDK/Web Component versionado (npm package).
2. Interfaz abstracta `ModelProvider` para cambiar de proveedor.
3. Temas personalizables vía CSS variables.
4. Documentación de integración para otros proyectos.

12. Widget de chat — especificación

12.1 Integración en Sistema R

El widget se integra en Sistema R como un **Web Component** nativo (`<sistema-r-chat>`) o como un componente React. Se añade al layout principal de la app (`frontend/app/layout.tsx` O `frontend/components/shell-frame.tsx`):

```
// Ejemplo de integración como Web Component
// 1. Cargar el script (solo cuando CHAT_IA_ENABLED=true)
<script src="https://cdn.example.com/sistema-r-chat/v1/widget.js"></script>

// 2. Usar el componente (pasa JWT del usuario actual)
<sistema-r-chat
  jwt="eyJ..."
  gateway-url="https://.../v1/chat"
  role="admin"
  lang="es"
></sistema-r-chat>
```

Feature flag: Solo renderizar cuando `CHAT_IA_ENABLED=true` y el rol del usuario es `admin` o `aunor`.

Estilo: Botón flotante en la esquina inferior derecha. Panel lateral (400px de ancho en desktop, full-width en mobile). CSS personalizable mediante variables: `--chat-primary-color`, `--chat-bg`, `--chat-font-family`.

12.2 Atributos/props del widget

Prop	Tipo	Descripción
jwt	string	JWT de Supabase Auth del usuario actual (obligatorio)
gateway-url	string	URL base del Gateway (obligatorio)
role	string	Rol del usuario: 'admin' 'aunor'
lang	string	Idioma: 'es' por defecto
theme	object	Opcional: colores, border-radius, fuente
on-error	(error) => void	Callback de error

12.3 Responsabilidades del widget

- Mantener el JWT actualizado (refrescar cuando Supabase renueve el token).
- Enviar mensajes al Gateway: { jwt, messages: [{role, content}] }.
- Renderizar streaming de chunks: tokens (texto), tool_calls (mostrar qué tool se llamó), tool_results (mostrar datos ejecutados).
- Mostrar distinción entre "Dato de la plataforma" e "Inferencia del modelo".
- Persistir conversación localmente (localStorage) como respaldo si el Gateway no responde.
- Indicador de estado: conectando, escribiendo, error.
- Soporte para Markdown básico en respuestas (negritas, listas, tablas).

13. Estructura del repositorio recomendada

```
sistema-r-chat-ia/
├── gateway/
│   ├── src/
│   │   ├── index.ts           # Entry point (Edge Function o serverless)
│   │   ├── auth.ts           # JWT validation + session check
│   │   ├── router.ts         # Endpoint routing
│   │   ├── chat.ts           # Chat handler (streaming, tools, model)
│   │   └── tools/
│   │       ├── index.ts      # Tool registry
│   │       ├── admin-read.ts # 14 read tools for Admin
│   │       ├── admin-write.ts # 14 write tools for Admin
│   │       ├── aunor-read.ts # 8 read tools for Aunor
│   │       └── definitions.ts # JSON Schema definitions for OpenAI
│   ├── provider/
│   │   ├── interface.ts     # ModelProvider interface
│   │   └── openrouter.ts     # OpenRouter implementation
│   ├── usage.ts             # Spending limits + chat_usage writes
│   └── prompt.ts            # System prompt builders per role
├── supabase/
│   └── migrations/          # Chat tables (chat_sessions, chat_messages, chat_usage)
├── tests/
│   ├── golden-set.json     # Evaluación de preguntas
│   ├── auth.test.ts
│   ├── tools.test.ts
│   ├── isolation.test.ts   # Tests de RLS entre roles
│   └── spending.test.ts
├── .env.example
└── package.json

└── widget/
    ├── src/
    │   ├── index.ts         # Web Component registration
    │   ├── chat-widget.ts   # Main chat component
    │   └── chat-message.ts  # Message renderer
```

```
| | | chat-input.ts      # Input + send
| | | streaming.ts      # SSE/streaming parser
| | | styles.css        # Theme variables
| | package.json
| | vite.config.ts
|
| docs/
| | ESPECIFICACION.md  # Este documento
| | CHANGELOG.md
|
| .github/
| | workflows/
| | | ci.yml           # Typecheck + tests + lint
| | | deploy.yml       # Deploy gateway (Supabase o Vercel)
|
| README.md
```

14. Pruebas requeridas

14.1 Tests unitarios del Gateway

Categoría	Casos de prueba
Autenticación	• JWT inválido → 401 • JWT expirado → 401 • JWT válido pero sin sesión app_sessions activa → 403 • JWT de usuario con must_change_password=true → 403 con mensaje • JWT de Admin → tools de escritura disponibles • JWT de Aunor → solo tools de lectura
Tools Admin (lectura)	• list_activities con filtros devuelve resultados correctos • get_activity_detail devuelve actividad + spans + auditoría • get_trash devuelve solo actividades con deleted_at NOT NULL • search_activities busca por título y descripción
Tools Admin (escritura)	• create_activity crea actividad y devuelve id+version • create_activity con idempotency_key repetido devuelve misma actividad • soft_delete_activity con motivo válido → deleted_at no nulo • soft_delete_activity SIN confirmación → debe fallar • advance_activity con version incorrecta → error de concurrencia
Tools Aunor	• my_activities devuelve solo actividades publicadas para Aunor • my_activity_detail incluye deliveries y journeys • my_deliveries devuelve entregas con estado de confirmación
Control de gasto	• Request con >50k tokens → 413 • Request que excede tope diario → 429 • Modelo no permitido en lista blanca → usa modelo por defecto
Rate limiting	• >30 requests/min → 429 • Reset de contador después de 1 minuto

14.2 Tests de aislamiento de roles

CRÍTICOS: Verificar que la IA respeta exactamente las mismas RLS que la interfaz web.

Escenario	Resultado esperado
Admin pregunta por actividades de un operario específico	Admin puede ver todas las actividades (incluyendo las de otros operarios)
Aunor intenta ver la tool list_operators	Tool no está disponible para Aunor (no declarada en system prompt)
Aunor pregunta "¿qué opina el operario de la actividad?"	La tool aunor_activities no expone operator_opinion. IA responde "información no disponible"
Aunor intenta ejecutar create_activity	Tool no disponible. IA responde "El espacio Aunor es de solo lectura."
Admin consulta aunor_deliveries	Admin puede ver entregas Aunor (misma vista que Aunor, pero Admin tiene permisos más amplios)
Aunor consulta audit_events de una actividad	Vista aunor_activities no expone auditoría interna. IA responde que no tiene acceso.

14.3 Tests de integración con Sistema R real

- Ejecutar cada tool contra un proyecto Supabase de prueba (base UUID desechable).
- Verificar que el resultado de la tool coincide con una consulta directa a la base de datos.
- Verificar que las escrituras se reflejan correctamente (leer después de escribir).
- Verificar que las RPCs fallan correctamente con versiones incorrectas.

14.4 Evaluación de calidad del modelo (golden set)

Métrica	Objetivo	Frecuencia
Precisión en respuestas factuales	>95%	Semanal
Ejecución correcta de tools	>98%	Por despliegue
Tasa de alucinaciones	<2%	Semanal
Latencia p50	<3s para respuestas simples	Continua
Latencia p95	<10s para respuestas con tools	Continua
Coste por conversación promedio	<\$0.05	Continua
Coste mensual total	<\$50	Mensual

15. Definition of Done — Fase 1 (MVP)

- Gateway desplegado y accesible (POST /v1/chat responde 200 con streaming).
- Widget integrado en Sistema R con feature flag CHAT_IA_ENABLED, visible solo para Admin y Aunor.
- Admin puede: listar actividades, ver detalle, buscar, consultar histórico, ver operarios (5 tools de lectura).
- Aunor puede: consultar sus actividades, ver detalle, ver entregas, buscar material, ver calendario (5 tools de lectura).
- Las tools de escritura existen en la declaración pero el Gateway las ignora (CHAT_IA_WRITE_ENABLED=false).
- Control de gasto implementado: límite de tokens/request, USD/día, USD/mes, lista blanca de modelos.
- La clave OPENROUTER_API_KEY solo existe como variable de entorno del Gateway.

- Tests de aislamiento: verificado que Aunor no puede ver datos internos.
- Sin modificaciones al código de Sistema R existente (el widget se añade como script externo).

16. Objetivos del Chat IA

16.1 Objetivo general

Implementar un asistente conversacional que permita a **Admin** ejecutar acciones administrativas sobre Sistema R mediante lenguaje natural (reduciendo la carga manual de ingreso de datos), y a **Aunor** consultar sus datos publicados con respuestas contextuales y precisas sin intervención de Admin.

16.2 Objetivos específicos

- Automatización Admin:** Reducir en un 80% el tiempo de carga manual de datos del equipo a la plataforma. El Admin describe en lenguaje natural qué actividades crear, y la IA las materializa en la base de datos.
- Autoservicio Aunor:** El cliente obtiene respuestas sobre actividades, entregas, acuerdos y reemplazos sin esperar a que Admin responda.
- Trazabilidad:** Cada acción ejecutada por la IA queda registrada en auditoría con el actor real (el Admin autenticado).
- Aislamiento:** La IA nunca expone información que el rol del usuario no tenga permiso de ver (mismas RLS que la interfaz web).

16.3 Alcance de la Fase 1 (MVP)

Incluye	No incluye (futuro)
• 10 tools de lectura (5 Admin + 5 Aunor)	• Tools de escritura (Fase 2)
• Gateway con validación JWT y streaming	• Fine-tuning de modelos
• Widget embebido en Sistema R	• RAG sobre documentos externos (PDFs, contratos)
• Control de gasto (topes, lista blanca, métricas)	• Subida de archivos por chat
• Distinción dato vs inferencia	• Agentes autónomos sin supervisión
• Tests de aislamiento de roles	• WhatsApp / Slack / notificaciones
• Feature flag para activar/desactivar	• SDK reutilizable para otros proyectos

16.4 Contexto del usuario Marco (Admin)

Necesidad principal: "El equipo me manda data y yo la implemento en la plataforma. Quiero que una IA lo haga por mí."

Marco es el Admin de Sistema R. Su flujo actual implica recibir datos del equipo (actividades, jornadas, entregas, acuerdos con Aunor) e ingresarlos manualmente en la plataforma. El Chat IA debe permitirle decir: "Crea una actividad de Grabación para Ana, 'Cobertura Norte', del 5 al 7 de octubre, lugar Caseta Norte" y que la IA ejecute `plan_activity_v2` automáticamente.

Adicionalmente, Marco necesita consultar el estado general de la plataforma: "¿Cuántas actividades entregó Ana este mes?", "¿Qué hay en la papelera?", "Muéstrame las publicaciones pendientes de confirmación de Aunor."

17. Referencias del repositorio original

Archivo en Sistema R	Propósito
supabase/migrations/202608220001_backend_foundation.sql	Tablas base (profiles, app_sessions, activities, activity_date_spans, audit_events), RLS policies, RPCs de sesión, helpers de seguridad
supabase/migrations/202608280001_activity_authority.sql	RPCs de actividades (plan_activity_v1, create_own_activity_v1, update_execution_v1, advance_activity_v1, soft_delete_activity_v1, restore_activity_v1)
supabase/migrations/202608300002_private_conversations.sql	Tabla activity_messages, RPCs post_activity_message_v1, edit, delete
supabase/migrations/202609060001_aunor_role.sql	Rol aunor + schema private con tablas Aunor
supabase/migrations/202609060002_aunor_space.sql	RPC aunor_mutate_v1 + vistas aunor_*
supabase/migrations/202609110003_restart_activity.sql	RPC restart_activity_v2
supabase/migrations/202609110004_explicit_erasure.sql	RPCs de erasure (preview, execute, cancel)
supabase/functions/admin-accounts/index.ts	Edge Function para crear cuentas y resetear passwords
supabase/functions/change-temporary-password/index.ts	Edge Function para cambio de clave temporal
supabase/functions/admin-erasure/index.ts	Edge Function para borrado físico
docs/contrato-producto-vigente-2026-08-28.md	Contrato de producto (sección 10: Chat IA separado)
docs/plan-chat-ia-reutilizable.html	Plan original del Chat IA (3 páginas)

18. Variables de entorno requeridas

Variable	Descripción	Ejemplo
SUPABASE_URL	URL del proyecto Supabase de Sistema R	https://xxx.supabase.co
SUPABASE_SERVICE_ROLE_KEY	Service role key (solo para Auth admin + RPCs que requieren service_role)	
OPENROUTER_API_KEY	API key de OpenRouter	
CHAT_IA_ENABLED	Feature flag: true/false	true
CHAT_IA_WRITE_ENABLED	Habilita tools de escritura (Fase 2+)	false (Fase 1)
ALLOWED_MODELS	JSON array de modelos permitidos	["anthropic/claude-3.5-haiku", "openai/gpt-4o-mini"]

ACTION_MODEL	Modelo para acciones de escritura	anthropic/claude-sonnet-4
MAX_TOKENS_PER_REQUEST	Límite de tokens por request	50000
MAX_RESPONSE_TOKENS	Límite de tokens de respuesta	4096
DAILY_SPENDING_LIMIT	Tope de gasto diario en USD	5
MONTHLY_SPENDING_LIMIT	Tope de gasto mensual en USD	50